



Graph-Based Models

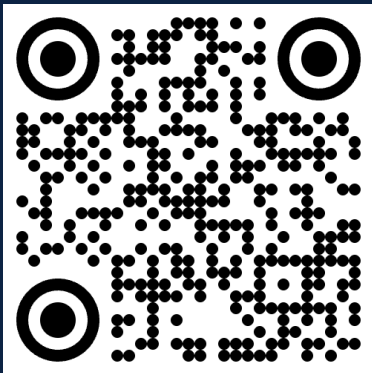
The Advanced R Series: Clustering & Classification

Slides and code created by Jesse Ghashti.

February 23, 2026

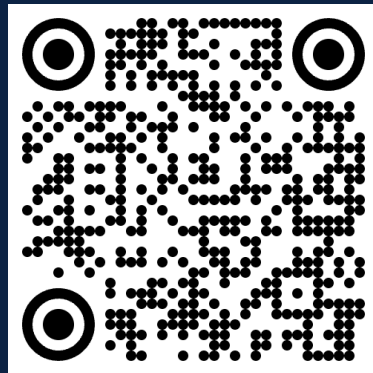
Centre for Scholarly Communication

The University of British Columbia | Okanagan Campus | Syilx Okanagan Nation Territory



←
Check out our other
CSC workshops!

GitHub code and
slides for today's
workshop (and pre-
vious workshops)



Alternatively, code/slides available at the bottom of
<https://csc-ubc-okanagan.github.io/workshops/>

Workshop Series Overview



Session	Topic	Date/Time
1	Hierarchical Models	Jan 14, 12:00 PM
2	Centroid-Based Models	Jan 19, 3:30 PM
3	Fuzzy Clustering	Jan 28, 1:00 PM
4	Distribution-Based Models	Feb 2, 3:30 PM
5	Density-Based Models	Feb 9, 3:30 PM
6	Graph-Based Models	Feb 23, 3:30 PM
7	Mixed-Type Data Quantification	Mar 4, 1:00 PM
8	Mixed-Type Data Clustering	Mar 11, 1:00 PM
9	Bias Reduction and Fairness	Mar 18, 1:00 PM
10	Dimensionality Reduction 1 of 2	Mar 23, 3:30 PM
11	Dimensionality Reduction 2 of 2	Mar 30, 3:30 PM

What we learned...

- Learned DBSCAN: core points, border points, and noise detection
- Used the k -distance plot to select ε
- Explored OPTICS and reachability plots for multi-scale clustering
- Introduced HDBSCAN with outlier scores and soft memberships
- Applied density-based methods to Chicago crime hotspot detection
- Compared DBSCAN vs HDBSCAN for varying density regions

Today -> Graph-Based Clustering



Today we will...

- Understand why some data is naturally relational (nodes and edges)
- Construct similarity graphs from coordinate data
- Learn the graph Laplacian and its spectral properties
- Use eigendecomposition and the eigengap for cluster selection
- Introduce modularity-based community detection (Louvain)
- Apply graph-based methods to infrastructure and airport networks

Today we require...

```
library('igraph')      # graph construction and algorithms
library('ggplot2')     # visualization
library('dplyr')       # data manipulation
library('gridExtra')   # plot arrangement
library('viridis')     # color palettes
library('Matrix')      # sparse matrix operations
```

From Coordinates to Connectivity

In previous workshops we clustered based on distances or densities.

But some data is naturally relational

- Social networks: people connected by friendships
- Infrastructure networks: substations connected by power lines
- Biological networks: proteins connected by interactions
- Transportation networks: airports connected by routes

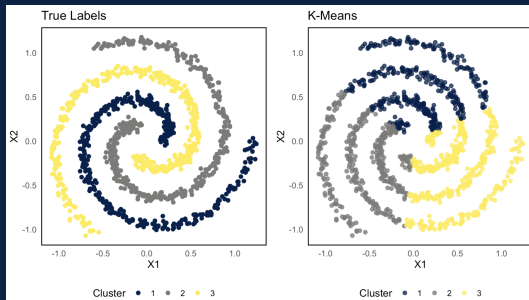
Graph-based clustering defines clusters based on topology

- Nodes tightly connected *internally*
- Loosely connected *externally*

K-means on Non-Convex Clusters



```
nSpiral <- 400
makeSpiral <- function(n, startAngle, noise = 0.03) {
  t <- seq(0, 2*pi, length.out = n)
  r <- 0.2 + t/(2*pi)
  data.frame(
    x = r*cos(t + startAngle) + rnorm(n, 0, noise),
    y = r*sin(t + startAngle) + rnorm(n, 0, noise))
}
spiral1 <- makeSpiral(nSpiral, 0)
spiral2 <- makeSpiral(nSpiral, 2*pi/3)
spiral3 <- makeSpiral(nSpiral, 4*pi/3)
spiralDf <- rbind(
  data.frame(spiral1, trueCluster = factor(1)),
  data.frame(spiral2, trueCluster = factor(2)),
  data.frame(spiral3, trueCluster = factor(3)))
kmeansSpirals <- kmeans(spiralDf[, c("x", "y")],
  centers = 3, nstart = 25)
```



K-means fails because spirals interweave non-convexly. Even DBSCAN struggles here due to uniform density throughout.

Graphs and Similarity

A graph $G = (V, E)$ consists of:

- V : a set of nodes (the observations)
- E : a set of edges (pairwise relationships)

Weighted similarity matrix $\mathbf{W}$:

- W_{ij} = similarity between nodes i and j
- $W_{ij} = 0$ means no edge between i and j

Graph construction options:

- ε -neighbourhood: connect i, j if $d(i, j) < \varepsilon$
- k -NN graph: connect i to its k nearest neighbours
- Fully-connected: no trimming (computationally expensive)

SPECTRAL CLUSTERING -> Step 1: Build the Similarity Matrix

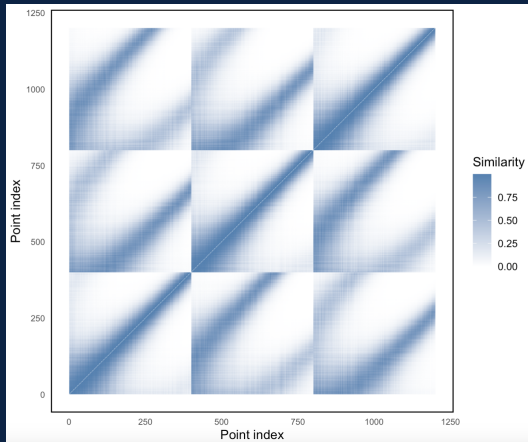


```
spiralMatrix <- as.matrix(spiralDf[, c("x", "y")])  
nPts <- nrow(spiralMatrix)  
distMatrix <- as.matrix(dist(spiralMatrix))  
sigma <- 0.5  
W <- exp(-distMatrix^2 / (2*sigma^2))  
diag(W) <- 0
```

Gaussian (RBF) kernel:

$$W_{ij} = \exp\left(-\frac{d(i,j)^2}{2\sigma^2}\right)$$

- σ controls the width of the kernel
- Nearby points $\rightarrow$ high similarity
- Distant points $\rightarrow$ similarity ≈ 0



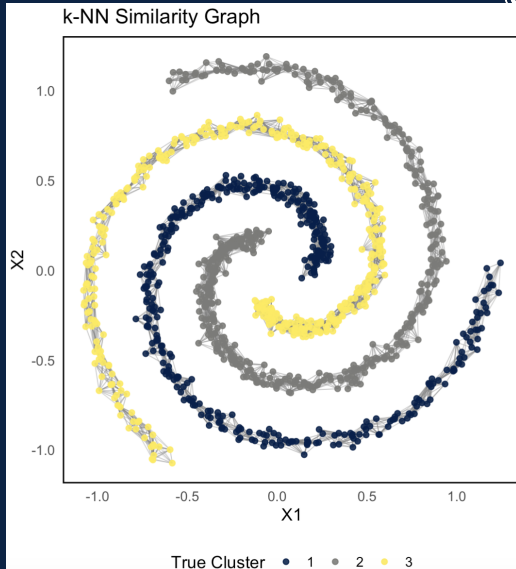
Step 2 -> Sparsify to k -NN Graph

```
knn <- 10
Wsparse <- matrix(0, nrow = nPts, ncol = nPts)

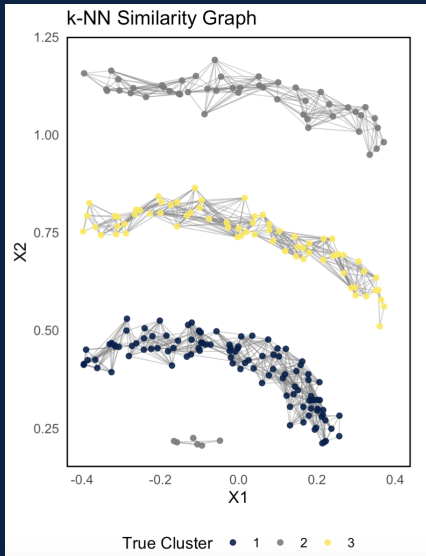
for (i in seq_len(nPts)) {
  topK <- order(W[i, ], decreasing = TRUE)[1:knn]
  Wsparse[i, topK] <- W[i, topK]
}

Wsparse <- pmax(Wsparse, t(Wsparse))
gSpirals <- graph_from_adjacency_matrix(
  Wsparse, mode = "undirected",
  weighted = TRUE, diag = FALSE)
```

A fully-connected $\mathbf{W}$ has n^2 edges. Keeping only the k strongest connections per node reduces complexity while preserving local structure.



k -NN Graph (Zoomed)



Each point connects primarily to its neighbours *along the same spiral*, not across spirals. This local connectivity encodes cluster structure.

Step 3 -> Compute the Graph Laplacian



The Graph Laplacian

Degree matrix **D**: diagonal matrix where $D_{ii} = \sum_j W_{ij}$

Unnormalized Laplacian

$$\mathbf{L} = \mathbf{D} - \mathbf{W}$$

Normalized (symmetric) Laplacian¹

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{W} \mathbf{D}^{-1/2}$$

Properties

- **L** is positive semi-definite (eigenvalues ≥ 0)
- Number of zero eigenvalues = number of connected components
- Small eigenvalues reveal cluster structure

[1]: Shi, J. and Malik, J., 2000. Normalized cuts and image segmentation. *IEEE Transactions on pattern analysis and machine intelligence*, 22(8), pp.888-905.

```
dVec <- rowSums(Wsparse)
D <- diag(dVec)
L <- D - Wsparse
dInvSqrt <- diag(1 / sqrt(pmax(dVec, 1e-10)))
Lsym <- dInvSqrt %*% L %*% dInvSqrt
```

Why normalize?

- Accounts for varying node degrees
- More robust to outliers and degree heterogeneity
- Eigenvectors are better behaved for clustering

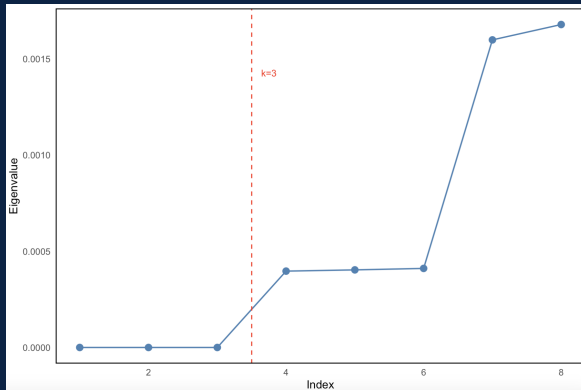
Step 4 -> Eigendecomposition and the Eigengap



```
eigenLsym <- eigen(Lsym, symmetric = TRUE)
eigenvalues <- rev(eigenLsym$values)
eigenvectors <- eigenLsym$vectors[,
  rev(seq_len(ncol(eigenLsym$vectors)))]
eigenDf <- data.frame(
  index = 1:8,
  eigenvalue = eigenvalues[1:8])
```

The Eigengap Heuristic

A large jump between λ_k and λ_{k+1} suggests k clusters. Similar to the elbow method for WCSS or the k -distance plot for DBSCAN.



Step 5 -> Spectral Embedding



Embedding Nodes into $\mathbb{R}^k$

The first k eigenvectors of $\mathbf{L}_{\text{sym}}$ embed the n nodes into $\mathbb{R}^k$:

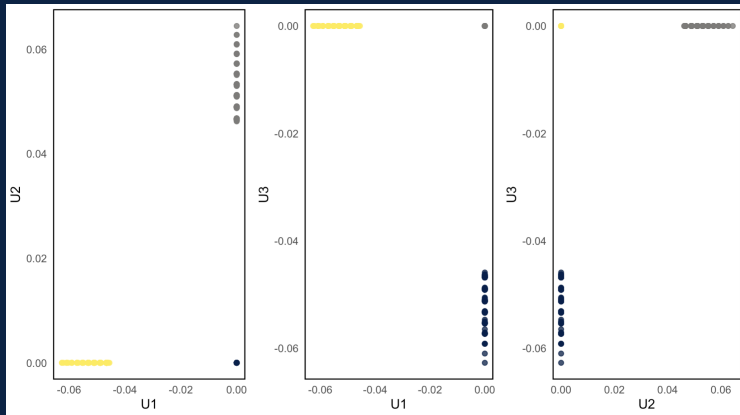
$$\mathbf{U} = [\mathbf{u}_1 \mid \mathbf{u}_2 \mid \cdots \mid \mathbf{u}_k] \in \mathbb{R}^{n \times k}$$

Each row of $\mathbf{U}$ is a k -dimensional representation of a node.

In this embedding, nodes from the same cluster are mapped close together, even if they were intertwined in the original space.

```
k <- 3
U <- eigenvectors[, 1:k]
embedDf <- data.frame(u1 = U[, 1], u2 = U[, 2], u3 = U[, 3],
                      trueCluster = spiralDf$trueCluster)
```

Visualizing the Spectral Embedding



The intertwined spirals become separable in the eigenspace! Standard clustering methods now work.

Step 6 -> Cluster in the Eigenspace

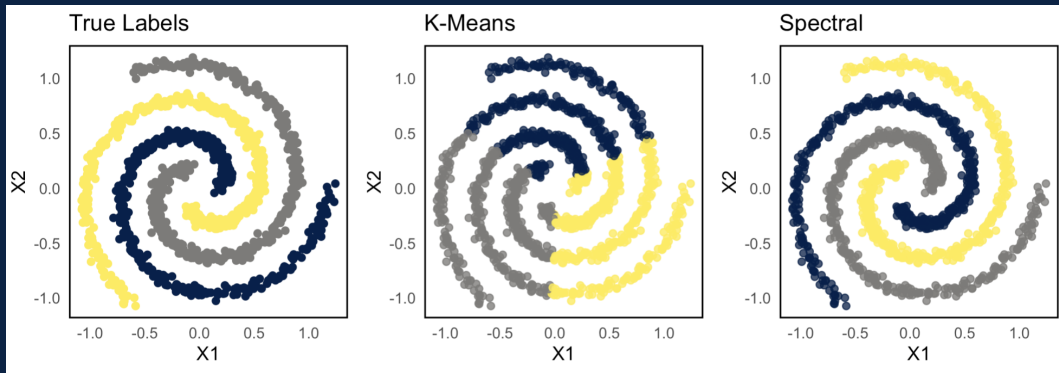


```
rowNorms <- sqrt(rowSums(U^2))
rowNorms[rowNorms == 0] <- 1
T <- U / rowNorms
spectralKmeans <- kmeans(T, centers = k, nstart = 50)
spiralDf$spectralCluster <-
  factor(spectralKmeans$cluster)
```

Row normalization

Projects each row onto the unit sphere. This ensures that clustering is based on *direction* in eigenspace, not magnitude. Then we apply K -means in the transformed space.

True Labels vs K -means vs Spectral



Step	Operation
1	Build similarity matrix $\mathbf{W}$ (e.g., Gaussian kernel)
2	Sparsify to k -NN graph (optional but recommended)
3	Compute normalized Laplacian $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$
4	Compute first k eigenvectors of $\mathbf{L}_{\text{sym}}$
5	Form matrix $\mathbf{U} \in \mathbb{R}^{n \times k}$ from eigenvectors
6	Row-normalize $\mathbf{U}$ to get $\mathbf{T}$
7	Apply K -means to rows of $\mathbf{T}$

Modularity Q

Modularity measures how much edge density within communities exceeds what we would expect under a random graph with the same degree sequence:

$$Q = \frac{1}{2m} \sum_{ij} \left(W_{ij} - \frac{d_i d_j}{2m} \right) \delta(c_i, c_j)$$

- m = total edge weight
- d_i = degree of node i
- c_i = community of node i
- $\delta(c_i, c_j) = 1$ if $c_i = c_j$, else 0

We want to find a partition that maximizes Q .

Fast Modularity Optimisation

Phase 1: Local moves

1. Start with each node in its own community
2. For each node, compute ΔQ for moving to each neighbour's community
3. Move to the community with largest $\Delta Q > 0$ (if any)
4. Repeat until no improvement

Phase 2: Aggregation

1. Aggregate communities into single “super-nodes”
2. Build a new graph on super-nodes
3. Repeat Phase 1 on the new graph

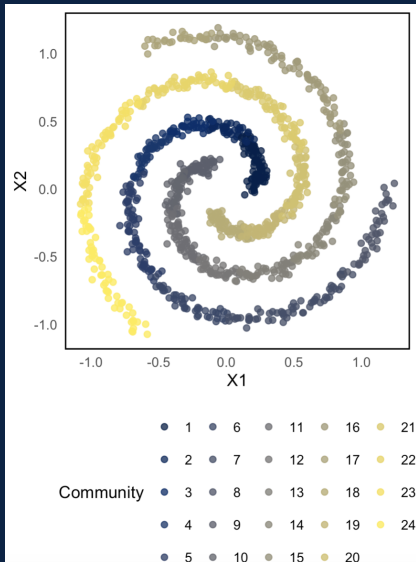
This produces a hierarchy of communities.

Louvain on the Spiral Data



```
louvainResult <- cluster_louvain(gSpirals)
spiralDf@louvainCluster <-
  factor(membership(louvainResult))
round(modularity(louvainResult), 4)
max(membership(louvainResult))
```

Louvain on the k -NN graph tends to over-partition because it finds locally optimal communities, not globally optimal ones. Spectral clustering with a fixed k often performs better when the true number of clusters is known.



Synthetic Power Grid Network

The scenario is a regional power grid with 150 nodes across 5 zones.

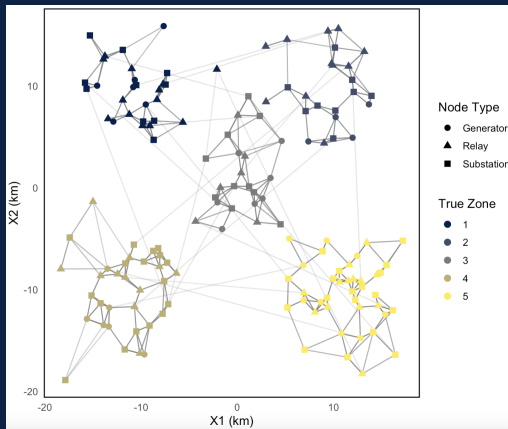
Nodes represent substations, relays, and generators.

Edges are transmission lines with capacity weights.

Our goal is to identify community structure that reflects operational zones.

Why graph-based?

- Network topology matters more than spatial coordinates
- Cross-zone connections are sparse (by design)
- Critical nodes (high betweenness) bridge communities



150 nodes, 319 edges. Five true zones with dense intra-zone connections and sparse inter-zone connections.

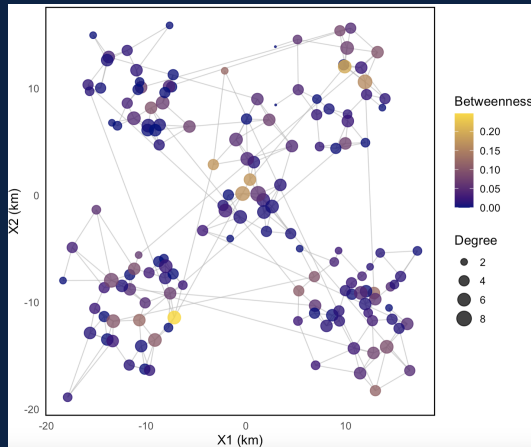
Graph Summary Statistics



```
nodeCoords$degree <- degree(gInfra)
nodeCoords$betweenness <-
  betweenness(gInfra, normalized = TRUE)
nodeCoords$strength <- strength(gInfra)
criticalNodes <- nodeCoords %>%
  arrange(desc(betweenness)) %>%
  head(10)
```

- **Degree** is number of connections
- **Betweenness** is fraction of shortest paths passing through node
- **Strength** is sum of edge weights

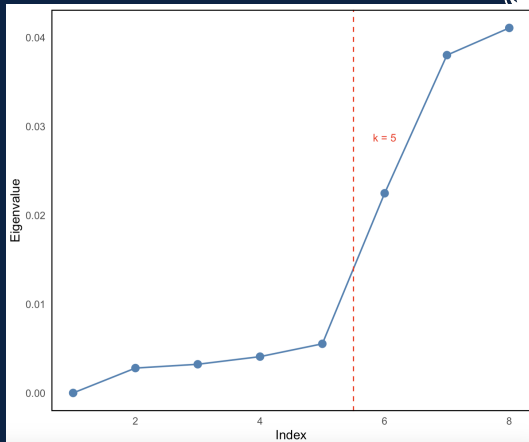
High betweenness → critical infrastructure.



Eigengap for Infrastructure Network



```
Winfra <- as_adjacency_matrix(gInfra,  
  attr = "weight", sparse = FALSE)  
dVecinfra <- rowSums(Winfra)  
dISinfra <- diag(1/sqrt(pmax(dVecinfra, 1e-10)))  
Lsyminfra <- dISinfra %*%  
  (diag(dVecinfra) - Winfra) %*% dISinfra  
eigeninfra <- eigen(Lsyminfra, symmetric = TRUE)  
eigvalsinfra <- rev(eigeninfra$values)  
eigvecsinfra <- eigeninfra$vectors[,  
  rev(seq_len(ncol(eigeninfra$vectors)))]
```

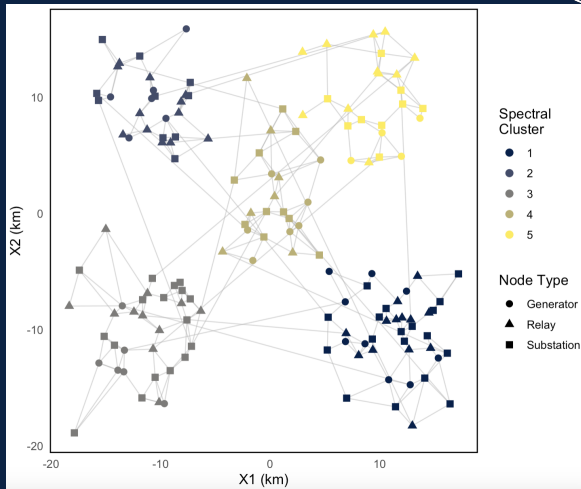


The eigengap suggests $k = 5$, matching the true number of zones.

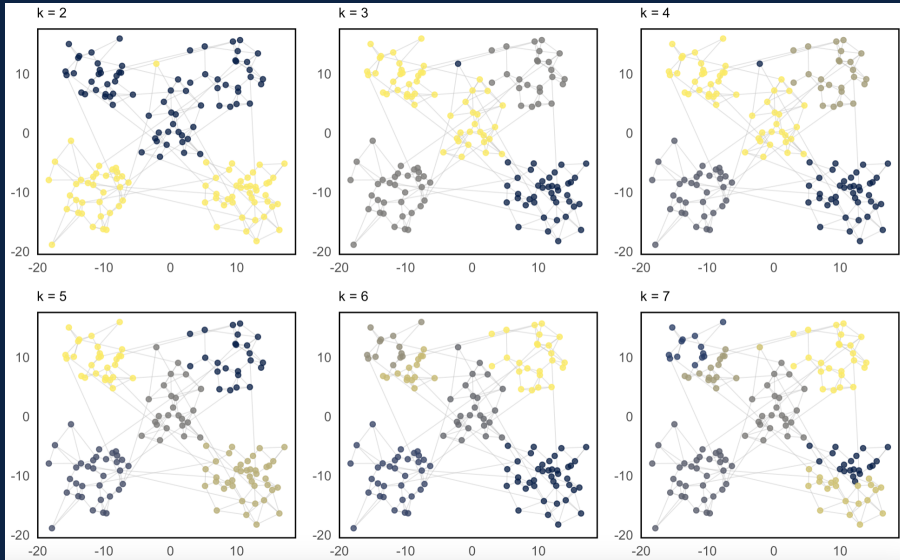
Spectral Clustering on Infrastructure



```
kInfra <- 5
Uinfra <- eigvecsinfra[, 1:kInfra]
rninfra <- sqrt(rowSums(Uinfra^2))
rninfra[rninfra == 0] <- 1
Tinfra <- Uinfra / rninfra
spectralInfra <- kmeans(Tinfra,
  centers = kInfra, nstart = 50)
nodeCoords$spectralCluster <-
  factor(spectralInfra$cluster)
```



Effect of k on Infrastructure Clustering

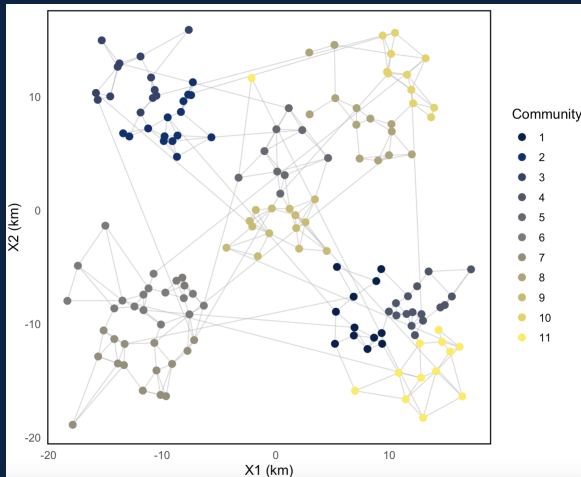


Louvain on Infrastructure Network

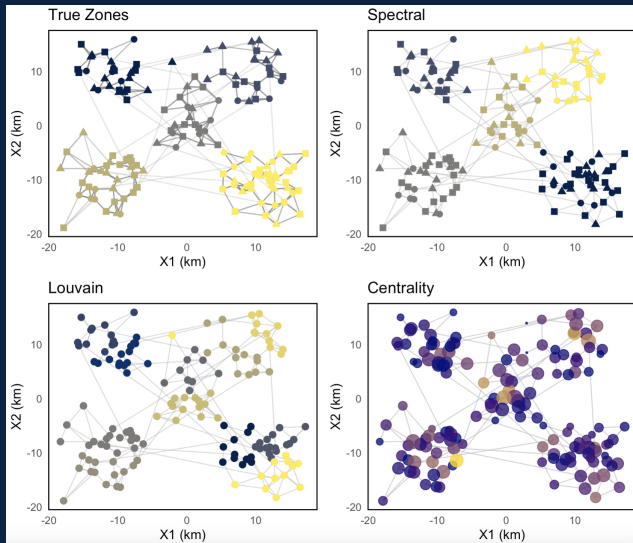


```
louvainInfra <- cluster_louvain(gInfra)
nodeCoords$louvainCluster <-
  factor(membership(louvainInfra))
max(membership(louvainInfra))
```

Louvain finds 11 communities, over-partitioning the 5 true zones. This is typical: Louvain optimizes modularity locally, which can fragment larger communities.



Infrastructure -> Comparison



louvainCluster	nNodes	meanDegree	maxBetween	nGenerators	nSubstations
1	12	3.75	0.1080	5	4
2	15	4.60	0.0958	2	5
3	13	3.85	0.0713	4	5
4	16	3.88	0.1275	1	8
5	9	5.00	0.1751	2	4
6	19	4.16	0.0891	1	9
7	15	4.60	0.2469	5	8
8	13	3.31	0.0726	3	5
9	14	5.14	0.1695	5	6
10	11	4.64	0.1973	1	4
11	13	4.08	0.1241	3	6

```
trueLabelsInfra <- as.numeric(nodeCoords$trueZone)
ariSpectral <- computeARI(trueLabelsInfra,
  as.numeric(nodeCoords$spectralCluster))
nmiSpectral <- computeNMI(trueLabelsInfra,
  as.numeric(nodeCoords$spectralCluster))
ariLouvain <- computeARI(trueLabelsInfra,
  as.numeric(nodeCoords$louvainCluster))
nmiLouvain <- computeNMI(trueLabelsInfra,
  as.numeric(nodeCoords$louvainCluster))
```

- Spectral ARI: 0.986
- Louvain ARI: 0.539
- Spectral NMI: 0.982
- Louvain NMI: 0.788

Real Flight Route Data

Data source OpenFlights.org (airports.dat + routes.dat)

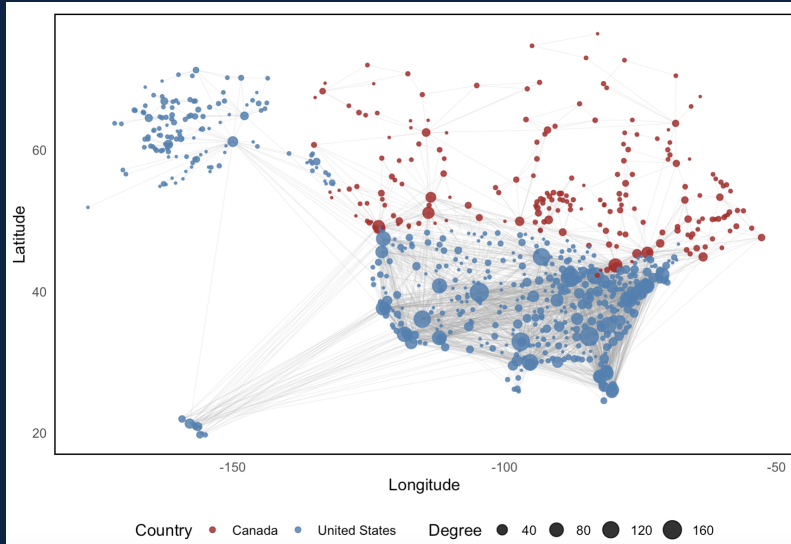
Scope is filtered to include only United States and Canada

Nodes include 745 airports (IATA codes)

Edges include 3,376 direct flight routes

Goal is discover regional structure from connectivity alone (no coordinates are used for clustering).

Airport Network by Country

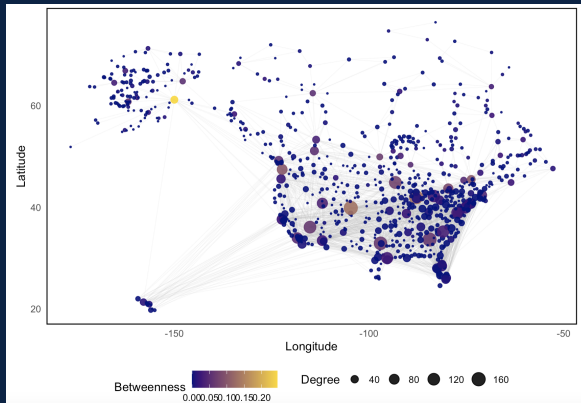


Airport Centrality



```
airportDf$degree <- degree(gAirport)
airportDf$betweenness <-
  betweenness(gAirport, normalized = TRUE)
```

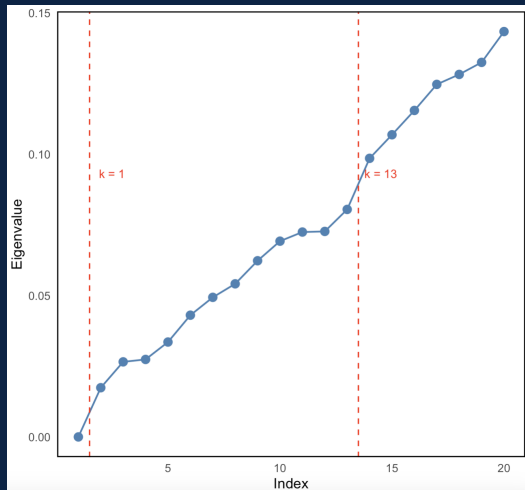
iata	city	country	degree	betweenness
ORD	Chicago	United States	162	0.13580050
DEN	Denver	United States	158	0.13393139
ATL	Atlanta	United States	155	0.06350023
DFW	Dallas-Fort Worth	United States	143	0.06467403
MSP	Minneapolis	United States	125	0.07679980
LAS	Las Vegas	United States	122	0.06358990
DTW	Detroit	United States	117	0.02757693
CLT	Charlotte	United States	113	0.03412391
IAH	Houston	United States	106	0.02766673
PHL	Philadelphia	United States	93	0.02246034
LAX	Los Angeles	United States	91	0.04679757
DCA	Washington	United States	89	0.01300612
SLC	Salt Lake City	United States	87	0.04028673
IAD	Washington	United States	85	0.02912142
EWK	Newark	United States	83	0.01552418



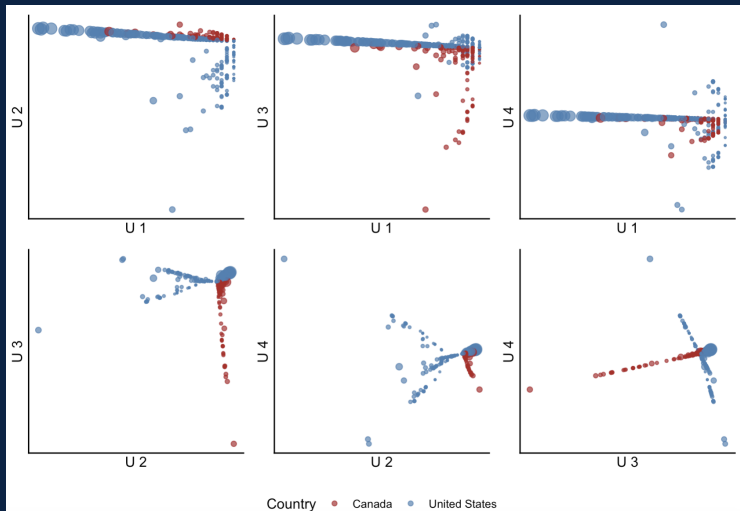
Eigengap for Airport Network



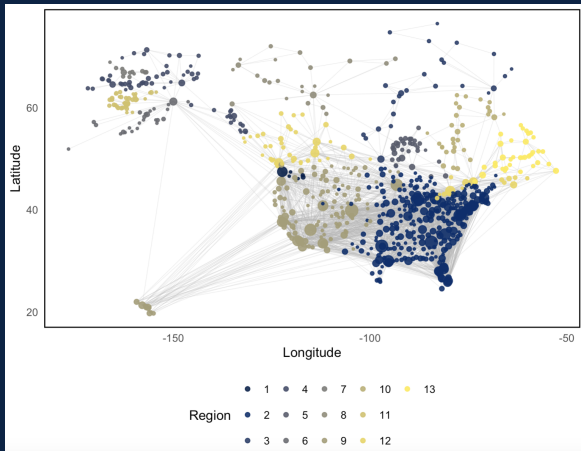
The eigengap shows potential cuts at $k = 1$ (trivial) and $k = 13$. We choose $k = 13$ to discover meaningful regional structure.



Spectral Embedding of Airports



Spectral Clustering on Airport Network

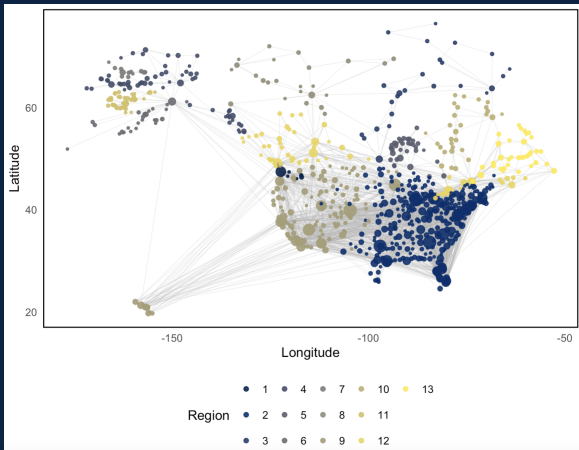


Spectral clustering discovers regional hubs purely from connectivity. Geographic proximity could be a byproduct of flight patterns.

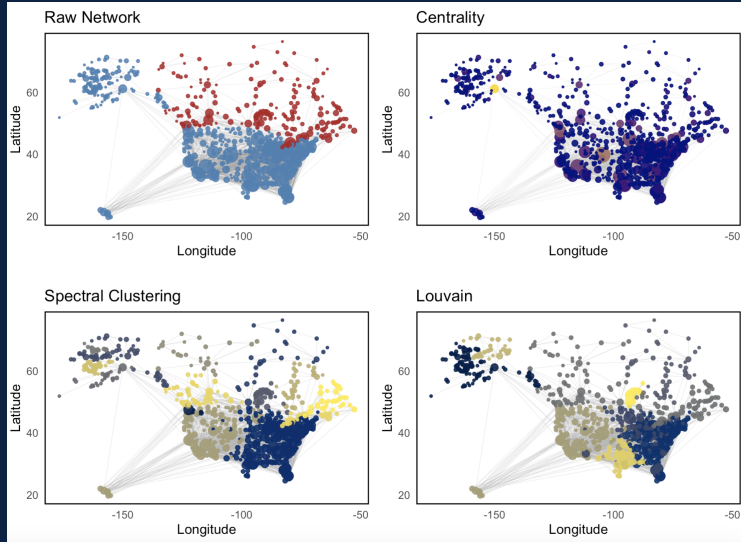
Spectral Graph Cuts



```
airportEdgeDf$clusterFrom <-  
airportDf$spectralCluster[airportEdgeDf$fromIdx]  
airportEdgeDf$clusterTo <-  
  airportDf$spectralCluster[airportEdgeDf$toIdx]  
airportEdgeDf$edgeType <- ifelse(  
  airportEdgeDf$clusterFrom ==  
    airportEdgeDf$clusterTo,  
    "Within cluster", "Cut edge")  
cutFracAir <- mean(  
  airportEdgeDf$edgeType == "Cut edge")  
round(cutFracAir, 3)
```



Airport Network -> Comparison



Spectral Cluster Profiles



spectralCluster	nAirports	nUS	nCanada	meanDegree	topHub	topHubCity
1	6	6	0	15.0	SEA	Seattle
2	281	281	0	13.4	ORD	Chicago
3	25	0	25	4.1	YWG	Winnipeg
4	58	58	0	3.2	FAI	Fairbanks
5	32	0	32	4.1	YXL	Sioux Lookout
6	25	25	0	3.0	ANC	Anchorage
7	12	12	0	3.0	OTZ	Kotzebue
8	24	0	24	3.1	YZF	Yellowknife
9	123	123	0	11.9	DEN	Denver
10	30	0	30	3.2	YVP	Quujjuaq
11	36	36	0	2.8	BET	Bethel
12	57	0	57	8.2	YYZ	Toronto
13	36	0	36	4.4	YQB	Quebec

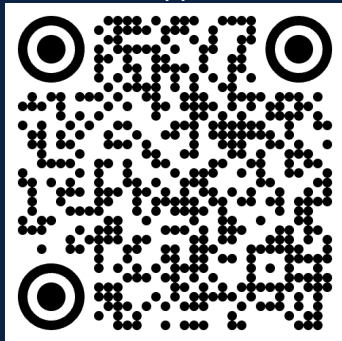
Spectral Clustering vs Louvain



	Spectral Clustering	Louvain
Requires k	Yes (eigengap)	No (auto-detected)
Objective	Min graph cut / Max embedding	Max modularity
Hierarchy	No (single level)	Yes (multi-level)
Over-partition risk	Low (fixed k)	High (local optima)
Best for	Known k , global structure	local structure

- Graph-based clustering uses connectivity, not just coordinates
- Similarity graphs can be built from distance data (Gaussian kernel + k -NN)
- Graph Laplacian encodes cluster structure in its spectrum
- Eigengap heuristic suggests the number of clusters (like elbow method)
- Spectral embedding transforms non-convex clusters into separable ones
- Modularity measures quality of community structure
- Louvain is fast and automatic but may over-partition
- Network metrics (degree, betweenness) can provide information on critical nodes
- Cut edges quantify how well clusters separate connectivity

Additional Questions?
Book an Appointment!



Next Workshop: Mar 4, 1:00 PM

Mixed-Type Data Quantification

- > So far we have clustered continuous (numeric) data.
- > Real datasets often have categorical, ordinal, and mixed features.
- > Next workshop will cover how to quantify mixed-type data for clustering.

Thank You!

Questions?

Workshop Materials:

<https://csc-ubc-okanagan.github.io/workshops/>

Contact:

jesse.ghashti@ubc.ca